

Evaluate Linger scenarios with Pydantic Logfire

Create a reviewed synthetic scenario, run it once, and record a review of the results using the sources and traces.

This guide follows the capture and bounded-curation example from 13 September 2026. The screenshots show real synthetic evaluation data. Use the same steps in your own configured Logfire project.

Follow the workflow

1 Choose objectives

Planning skill and report

2 Approve generation

Backstory and proposed answers

3 Review each answer

Local comparison frontend

4 Confirm one run

Adoption and evaluation

5 Inspect the evidence

Evals, cases, and traces

6 Record your decision

Findings and next comparison

Prepare the repository and destination

Open your Linger checkout in Codex with its Python environment installed. Set an explicit `LINGER_MODEL` and the matching provider key in the repository-root `.env`. The example model is `openai:gpt-5.6-luna`; your team can choose another supported model.

The supported provider prefixes are `openai:`, `google:`, and `anthropic:`. Their keys are `OPENAI_API_KEY`, `GOOGLE_API_KEY`, and `ANTHROPIC_API_KEY`. Keep key values in local configuration.

Configure Logfire from the repository root. Replace the placeholders with your own region, organization, and project before you run these commands.

```
uv run logfire --region <region> auth
uv run logfire --region <region> projects use \
--org <your-org> <your-project>
```

Local development can use the resulting project credentials. CI can use `LOGFIRE_TOKEN`. Synthetic traces contain scenario inputs and model exchanges, so choose the intended project before the run.

There are two approvals before the run. First approve the generation design. Later approve each proposed answer key. After the run, review the actual model outputs in Logfire.

Choose objectives and generate the scenario

- 1 **Invoke the planning skill.** Send this in Codex from the Linger repository.

Use `$plan-synthetic-scenarios` to choose evaluation objectives and write an implementation-oriented pre-generation report for review.

- 2 **Open the objective selector link.** Under **Choose what the evaluation should prove.**, select the Objectives you want to test. Hover or focus a card to read its scope and combination hints.

For this walkthrough, select **Reviewed automatic memory capture** and **Bounded memory curation**. Click **Confirm 2 objectives**. Return to Codex when the selector says that the agent has the validated selection.

- 3 **Read the pre-generation report.** The agent writes `pre-generation-report.md` in a new scenario directory. Check the intended behavior, planned Scenes, current implementation limits, and detached generator prompt.

Find new scenarios under `synthetic-journal-evaluation/scenarios/`. The [example planning report](#) shows the format; its implementation snapshot is historical.

Continue only when the prompt is marked **Runnable after human approval**, or its named preconditions have been resolved. **Target state - do not run** means the proposed design needs more work. Selector confirmation produces a report only.

- 4 **Approve the design and generation explicitly.** If the report is ready, send the following instruction. If it needs changes, describe them and review the revised report first.

I approve the runnable design and detached generator prompt in this report. Use that prompt to generate the two JSON files in its scenario directory, complete any required curation fields, and validate the Scenario. Stop for independent human review.

- 5 **Check the generated files.** Expect `backstory.json` and `ground-truth.json` beside the report. The first holds synthetic sources and Scenes. The second holds proposed answers and review criteria.

Generation is a separately authorized action. There is no separate generation skill in this workflow. The generator receives the detached prompt, while the agent completes evaluator-owned curation fields and validates both files.

A validator pass establishes structural and source consistency. The human review on the next page decides whether the proposed answers are correct. Use a new scenario directory for new data; the worked example already has an adoption record.

Approve every proposed answer key

Use `$review-synthetic-ground-truth` to review and adopt a validated synthetic scenario before replay. Use the files in `<scenario-directory>`.

Open the local review link returned by Codex. A human independent of the generator performs this review. The page title is **Read every source. Approve every answer key.**

06 scene-06 · reviewed_automatic_memory_capture

Capture candidate: allow capture

Needs changes ☐ **Approve this proposal**

01 Scene inputs
Everything available to this Scene before runtime.

02 Proposed Ground truth
The answer key that will become authoritative if approved.

1

2

LINE line-06

I was looking at the class timetable just now. I learn new crafts best in a small in-person class where I can ask questions as I go; with videos on my own, I tend to get stuck and give up.

Muse nomination **Capture Candidate**

Provenance decision **Allow Capture**

I learn new crafts best in a small in-person class where I can ask questions as I go; with videos on my own, I tend to get stuck and give up.

EXACT SOURCE SPANS

line-06 47-188

I learn new crafts best in a small in-person class where I can ask questions as I go; with videos on my own, I tend to get stuck and give up.

Green numbers match the steps on the same page: 1 compares the sources and proposed answer; 2 marks approval. Green means where to look or click, not that a result passed.

- 1 Read both columns for every Scene.** Check **Scene inputs**, **Proposed Ground truth**, the expected and prohibited outcomes, and any source spans or semantic review criteria. Use the numbered progress links to move between Scenes.
- 2 Approve or flag each proposal.** Select **Approve this proposal** only when the answer follows from the supplied sources. If it is wrong or ambiguous, click **Needs changes**.
- 3 Return corrections when needed.** Click **Make Changes**, then explain the affected Scene IDs and corrections in Codex. The agent revises and validates the files before a fresh review.

When all rows are approved and none are flagged, click **Confirm and run evaluation**. This records adoption and authorizes one supported provider run, which may incur model charges and send telemetry. The agent starts the run after validating the returned decision.

If the button instead says **Confirm Ground truth**, that exact selection supports adoption only. A valid combination in the selector does not guarantee an implemented replay.

Find your completed experiment

After confirmation, expect `ground-truth-adoption.json` beside the scenario files. It records who approved the exact input files. Editing either JSON file requires a new valid adoption before an adopted run.

The runner supplies the designated Scene inputs to the evaluated agents. The full Backstory and proposed or adopted answers stay outside their input. The grader uses the expected answers.

- 1 Wait for the agent to finish the run.** Keep the returned full evaluation JSON and the emitted Logfire link. The review frontend records the decision; the agent executes the supported replay.
- 2 Open your configured Logfire project.** Use the emitted link, or open **AI Evaluations** in the sidebar and choose **Datasets**. Select the dataset for your Objective or combination.
- 3 Open the experiment.** For this example, select `reviewed_capture_and_bounded_curation` and click **Review latest run**. To inspect an older run, use its row under **Recent experiments**.

reviewed_capture_and_bounded_curation Code-defined How it works Last 30 days

Last run about 16 hours ago

RECOMMENDED NEXT STEP

Review the latest run
Inspect cases from the latest run or compare it with another run.

Compare runs Review latest run

Cases Not reported Not available from this dataset source	Experiments 1 In the selected date range	Latest result Not reported No aggregate result was reported	Setup Defined in code Cases and evaluators included
---	--	---	---

Recent experiments
Open a run to review its cases, outputs, and evaluator evidence.

Experiment	When	Result	Actions
reviewed_capture_and_bounded_curation-24d1... 9 metadata fields	Latest Sep 13 11:46 PM	No aggregate result ...	Review

Green 3 marks Review latest run. Amber outlines flag interpretation cautions: "Not reported" in the dataset summary does not mean there is no experiment.

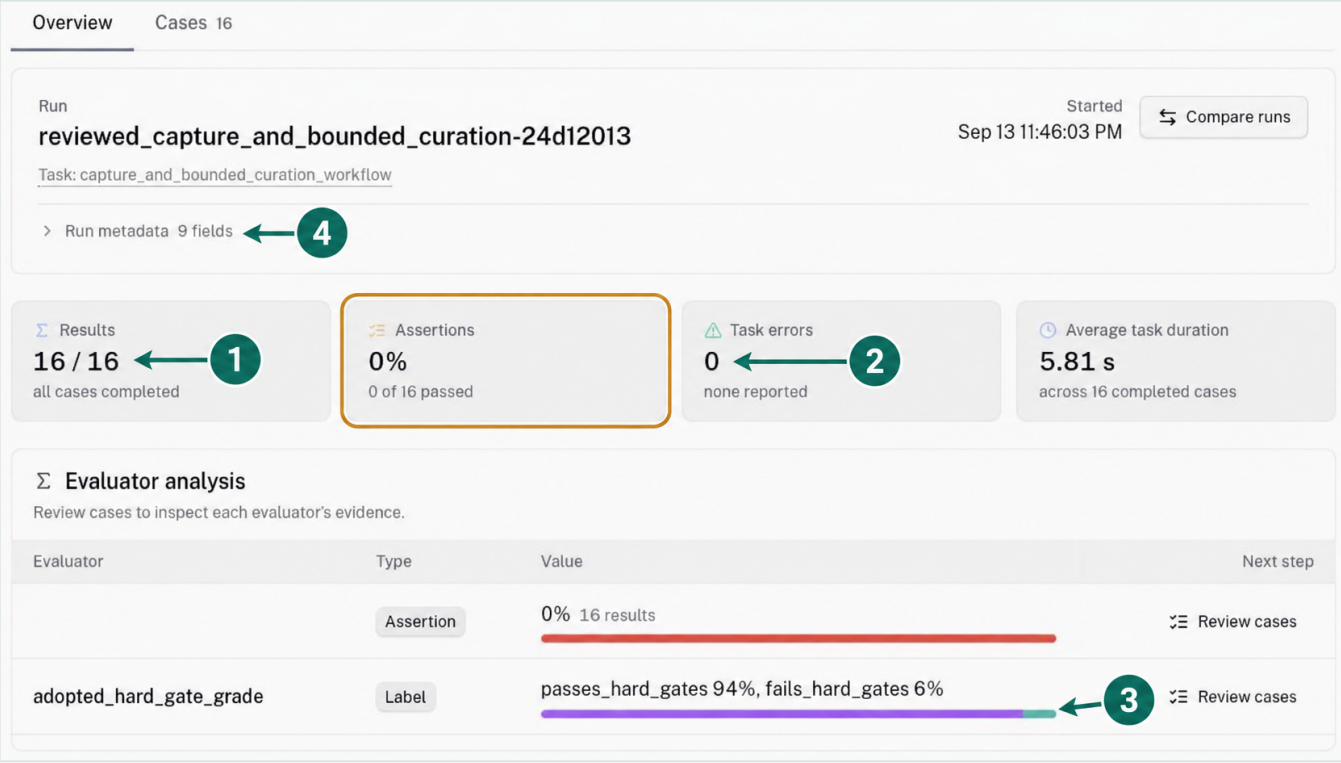
If the page is empty: check the organization, project, and date range first. A page named **compare** with no experiments can indicate an old malformed link. Return to your project's **Datasets** page. If no experiment was uploaded, inspect local configuration and the run log before authorizing another run.

Use your own project destination. The [worked-example dataset](#) requires access to Desmond Choy's Linger project; it is not a shared destination requirement.

The full JSON is the durable record. Direct review runs use a fresh temporary output path, so retain it in your team's normal evidence location. `$run-scenario` later saves run artifacts inside the scenario directory.

Read the overview without misreading the score

Open [the worked-example overview](#). A **dataset** is the set of cases. An **experiment** is one run of that dataset. Each **case** here corresponds to one Scene.



Match green markers 1-4 to the table below. The amber Assertions tile needs the explanation below the table.

1	Results: 16 / 16	All 16 cases completed. This count does not mean every case passed.
2	Task errors: 0	No task execution errors were reported. Output quality can still fail.
3	Grade label	The label records automated rule checks: 15 passes_hard_gates and 1 fails_hard_gates. Logfire rounds these to 94% and 6%.
4	Run metadata	Expand it to check ground_truth_status=adopted, the selected Objectives, run ID, and dataset_version.

Why does Assertions show 0%? This runner reports its grade as a string label, not a Boolean assertion. The current UI still shows an assertion aggregate. Use the label row and each case's hard_failures for this run. The empty **Failed** filter is misleading for the same reason.

Review duration, token use, and cost after correctness. These describe execution, not answer quality. Check the metric's aggregation before treating an average as a run total.

Inspect the failed grouping case

Click **Cases**, keep the **All** filter, and search for scene-15. Open the case. Click **Hide case list** to widen the comparison. Read **Input**, then **Expected output**, **Output**, and **Evaluator results**.

Selected case: **scene-15** Σ Scored

Show case list Open trace in Live view 15 of 16 Prev Next ×

INPUT

▶ {... 3 items}

OUTPUT

```

{ 7 items
  "response": { 2 items
    "kind": "no_curation_proposal",
    "reason": "These memories describe distinct routines and logistics-commute routing, changing clothes, bike maintenance, and posting concert flyers. They do not express the same durable memory or one evolving fact, and grouping them under a broad theme would not clearly improve retrieval."
  },
  "ground_truth_result": "fails_hard_gates",
  "hard_failures": [ 1 item
    0: "expected_curation_proposal"
  ],
  "semantic_review_required": true,
  "semantic_criteria": [... 1 item]
  "forbidden_semantic_claims": [... 1 item]
  "source_immutable": true
}

```

EXPECTED OUTPUT

```

{ 2 items
  "curation": { 2 items
    "primary_behavior": "related_topic_group",
    "expected": { 2 items
      "kind": "curation_proposal",
      "action": { 3 items
        "action": "assign_topic_group",
        "source_memory_ids": [ 3 items
          0: "prop-09",
          1: "prop-10",
          2: "prop-11"
        ],
        "semantic_review": {... 2 items}
      }
    },
    "ground_truth_status": "adopted"
  }
}

```

Evaluator results

adopted_hard_gate_grade · Label fails_hard_gates

Scene 15: 1 expands Input; 2 compares actual Output with Expected output; 3 opens the trace. Input is collapsed in this view.

- 1 Check which memories belong together.** The input contains a cycle route, a shirt kept at work, Sunday tyre checks, and a concert-flyer noticeboard. The first three support practical arrangements for cycling to work. The flyer record is a distractor.
- 2 Compare expected and actual behavior.** The adopted answer expects `assign_topic_group` for `prop-09`, `prop-10`, and `prop-11`. Sculptor returns `no_curation_proposal`. The recorded failure is `expected_curation_proposal`.
- 3 Record the mismatch, then inspect the trace.** Review whether the expected grouping is justified. If it is, investigate why Sculptor declined it. Treat the model's reason as evidence to inspect, not a proven root cause. [Open scene 15.](#)

The original sources stayed unchanged. No topic label was generated, so there is no generated label to assess for semantic quality in this failed case.

Check a passing summary for factual fidelity

Open [scene 14](#) from **Cases**. This case passes its deterministic checks, but `semantic_review_required=true` still asks for human judgment of the generated summary.

● OUTPUT

▼ { 7 items

▶ "response": { ... 2 items }

"ground_truth_result": "passes_hard_gates",

"hard_failures": []

"semantic_review_required": true,

▶ "semantic_criteria": [... 1 item]

▶ "forbidden_semantic_claims": [... 1 item]

"source_immutable": true

}

08

1

Automated grade. The hard checks passed.

2

Review still required. Judge the summary against its sources.

The response is collapsed. Green 1 marks the automated pass; green 2 marks the separate human review requirement.

Expand the summary and compare its timeline

Click the underlined, truncated summary in **Output** to show the full string. Read it alongside the three workbench source records in **Input**. The run produced this summary:

Folding workbench setup evolved from a planned spot under the living-room window (April 2026) to the hall alcove after direct afternoon sun made the window spot difficult for seeing parts (June 2026). Since August 2026, the hall-alcove setup has included a clamp lamp and works for small repairs, but the workbench still needs to be folded away when the drying rack is out.

April / prop-05	The window location was an untried plan.
June / prop-06	The trial exposed glare, and the workbench moved to the hall alcove.
August / prop-07	The hall setup has a clamp lamp and works for small repairs. The drying rack still requires the workbench to be folded away.

Confirm that the summary preserves this order, the current location, and the remaining constraint. Check that it excludes the unrelated bus-display record, prop-08. Reject invented causes, emotions, preferences, or conclusions. The summary limit is 100 words.

Record a separate human decision with the supporting source IDs. Approving the Ground truth before the run did not approve this generated summary. The recorded run still requires that semantic review.

Linger evaluation walkthrough | 14 Sep 2026

7 / 11

Review capture and correct no-change results

Open [scene 06](#) in **Cases**. Follow the memory candidate through review, binding, and storage.

Follow the capture stages in Output

<code>actual_capture_label</code>	Expect candidate. Muse found a candidate memory.
<code>capture.provenance_decision</code>	Expect <code>allow_capture</code> . Provenance accepted the proposed capture.
<code>capture.binding</code>	Expect <code>exact</code> . The reviewed words match the bound source span.
<code>capture.storage</code>	Expect <code>committed</code> . The application recorded the capture in the evaluation account.
<code>hard_failures</code>	Expect an empty list. Read any listed failure before relying on the pass label.

Compare the nominated span with the person's exact statement:

I learn new crafts best in a small in-person class where I can ask questions as I go; with videos on my own, I tend to get stuck and give up.

The opening comment about looking at the class timetable stays outside the memory. In the full `evaluation.json`, verify `stored_text`, `stored_record_count`, and `retry`. This run stored the exact sentence once. A repeated capture created no extra record and left the store unchanged.

Those storage and retry details are retained in the full JSON; the native Evals Output is a smaller projection. In Live, also read Muse's draft and Provenance's review to assess the reply and capture notice against the proposed outcomes.

Treat no change as a result to evaluate

Open scene -16. Its records use the word "key" for a door key, a key detail on a flyer, and a keyboard key. The expected and actual result is `no_curation_proposal`. Shared wording does not establish a useful relationship.

Review every Scene. In this run, capture Scenes 01-05 and 07-11 correctly store nothing. Scenes 12 and 13 propose duplicate links. Scene 14 needs summary review, scene 15 fails grouping, and scene 16 correctly leaves distinct records alone. The totals are 11/11 capture and 4/5 curation hard-check passes.

Curation tests proposal quality and preservation of the supplied originals. The captured memory from scene 06 is not fed into those curation cases. This run does not establish capture-triggered curation or later memory surfacing.

Open the trace to inspect the agent exchange

From a case, click **Open trace in Live view**. Find case: scene-15, then its sculptor.curation and Sculptor run spans. Select **Sculptor run** and open the **Agent Run** tab.

The screenshot shows the Linger Live view interface. On the left, a list of messages is displayed with timestamps. The right pane shows the details of a 'Sculptor run' message. The 'Input' section (user) contains a list of memories. The 'Output' section (assistant) contains a structured result object.

Input user

```
{
  "memories": [
    {
      "memory_id": "prop-09",
      "text": "For my cycle commute, I take the canal path because it avoids the busy junction beside the station."
    },
    {
      "memory_id": "prop-10",
      "text": "I leave a clean shirt at the arts centre so I can change after cycling to work."
    },
    {
      "memory_id": "prop-11",
      "text": "I check my bike tyres on Sunday evenings so the bike is ready for the week's commuting."
    },
    {
      "memory_id": "prop-12",
      "text": "The canal-side noticeboard is where I post the arts centre's public conce..."
    }
  ]
}
```

Output assistant

```
{
  "kind": "no_curation_proposal",
  "reason": "These memories describe distinct routines and logistics-commute_routing, changing clothes, bike mai..."
}
```

Green 1 points to the supplied memories in the exchange; green 2 points to the structured assistant result. Expand the blocks and scroll to read their full content.

- 1 Read the actual exchange.** Expand **Input system** with **See full text** to inspect the instructions. Read **Input user** to confirm which memories the agent received. Scroll to **Output assistant** for its structured response.
- 2 Interpret the structured result.** In scene 15, `final_result_NoCurationProposal` carries the no-change answer. "Final result processed" means that the structured answer was accepted by the agent framework; it is not proof that a memory was saved.

If Live shows only one case row: open the query editor and remove the `and span_id=...` condition while retaining `trace_id=...`. Click **Run** to execute the log search. This broadens the visible trace; it does not rerun the evaluation.

Use **Full Trace** or the expanded trace rows to locate child spans. Use **Metrics** for execution details and **Raw Data** for attributes. [Open this worked-example trace](#).

Use Agents and LLMs to explain execution

Set the date range to include the experiment. These project pages show execution metrics and can include other project activity.

Agents: find the role that produced the behavior

- 1 **Open an agent's traces.** In **Agents**, locate **Muse**, **Provenance**, or **Sculptor** and click that card's **Traces** link. Match the run and Scene before interpreting the exchange.

The screenshot shows three agent cards in the 'Agents' section. Each card has a header with a letter icon (P for Provenance, M for Muse, S for Sculptor) and the model name 'gpt-5.6-luna'. Below the header, there are four metrics: Runs, Cost, Avg Time, and Usage. Provenance has 22 runs, cost <\$0.01, avg time 1.7s, and usage is up. Muse has 11 runs, cost <\$0.01, avg time 3.7s, and usage is up. Sculptor has 5 runs, cost <\$0.01, avg time 2.9s, and usage is up. Each card also shows 'Last active 15h ago' and links for 'Optimize' and 'Traces'. A red circle with the number 1 and an arrow points to the 'Traces' link on the Sculptor card.

The example project shows 11 Muse runs, 22 Provenance runs, and 5 Sculptor runs in the selected time window. A Scene can invoke more than one agent.

LLMs and providers: inspect model calls

- 2 **Select the model.** Open **AI Inference**, choose **LLMs and providers**, then click **Models**. Select the model row to inspect calls, exceptions, latency, tokens, and cost.

The screenshot shows the 'LLMs and providers' page with the 'Models' tab selected. At the top, there's a search bar and a 'Docs' button. Below the search bar, there's a table with the following columns: Model, Calls, Exceptions, Latency, Speed, Input, Output, Cache read, Cost, Truncated, and Tool calls. The first row in the table is for the 'gpt-5.6-luna' model by OpenAI. A red circle with the number 2 and an arrow points to this row.

Model	↓ Calls	Exceptions	Latency	Speed	Input	Output	Cache read	Cost	Truncated	Tool calls
gpt-5.6-luna OpenAI	40	0.0% <0.001	2.3s max 7.0s	49 tok/s	122.1k	4.5k	92.6k	\$0.01	0.0% <0.001	Not reported

Models view. These are provider-call metrics, so 40 model calls can belong to 16 evaluation cases.

Use the individual trace to locate a long span, failed call, or truncated response. Return to **Evals** to check correctness against the adopted expected output. Zero exceptions and low cost do not establish that the behavior passed.

[Agents documentation](#) and [LLM documentation](#).

Compare a later run and record the decision

Run the saved scenario when you need new evidence

The reviewer's final button already starts the first supported run. For a later run, invoke the saved-scenario skill. To analyze an existing artifact, provide its path and say to use that saved run.

Use `$run-scenario` to choose a saved scenario, confirm its model, and run it with a Logfire link and an analysis of every Scene, including the reliability of passing results.

Select from the numbered menu, then confirm the exact provider and model. The skill checks prerequisites, runs all Scenes once, and saves the full JSON, summary, log, and analysis report under the scenario. It does not automatically retry a failed evaluation.

- 1 **Choose comparable experiments.** Open the dataset and click **Compare runs**. Add the new experiment, select one **Baseline**, then click **Apply comparison**. At least two runs are required.
- 2 **Compare changed cases.** Keep `dataset_version` and the grading rules fixed when testing a model or prompt change. Inspect each changed output, source selection, and failure. If the dataset changes, explain the difference before interpreting aggregate movement.

Save a review note with the evidence

Add your review to the run's analysis report and link any follow-up Beads issue.

Experiment / model / dataset_version:
 Scene ID and case or trace link:
 Expected behavior and supporting source IDs:
 Observed output and recorded hard failures:
 Human semantic decision: pass / fail / pending
 Reason and next action:
 Reviewer and date:

For the worked example, record the missed group in scene 15 and a separate semantic decision for scene 14. If you find a wrong answer key, revise and review a successor scenario. Preserve the original run and its recorded grade.

Sources and worked-example links

[Planning and generation](#)

[Example scenario and proposed answers](#)

[Ground truth reviewer](#)

[How this runner reports grades](#)

[Saved-scenario runs](#)

[Logfire datasets and experiments](#)

[Maintained evaluation workflow](#)

[Logfire experiment review](#)

Verified against Linger commit d9c91e5 and Logfire on 14 September 2026. Screenshot callouts were added with imagegen. The reviewer shows an unapproved local preview. UI labels and supported replay selections may change.